



TECHNICAL OVERVIEW

GPS Ushering and Events — How It's Built

Architecture, stack, and project layout — for a developer picking this up cold.

1. Stack

LAYER	CHOICE	WHY
Language / framework	Python 3 + FastAPI	Requested — the maintainer is Python/FastAPI-fluent, not JS
Templating	Jinja2 (via Starlette's <code>Jinja2Templates</code>)	Ships with FastAPI; real template inheritance (<code>{% extends %}</code> / <code>{% block %}</code>)
Content storage	Database rows (SQLAlchemy models), edited through the admin panel	No git/GitHub literacy required for the business owner to manage content
Database	SQLAlchemy 2.x ORM — SQLite locally, Postgres in production	Bookings plus all editable content now live in the same database
Auth (whole admin panel)	Single hardcoded admin user (env vars) + signed cookie session (<code>itsdangerous</code>)	One internal user (the business owner) — no need for a user table
Server	uvicorn (ASGI)	Standard FastAPI runtime
Client-side JS	Vanilla files under <code>app/static/js/</code> — <code>main.js</code> plus <code>location-picker.js</code>	Mobile nav, theme toggle, FAQ accordion, gallery filter/lightbox, booking form fetch, map picker — no framework, no build step
Maps	Leaflet + OpenStreetMap tiles + Nominatim reverse geocoding (all free, no API key)	Avoids requiring the client to set up a billed Google Cloud project just for a map

There is no Node.js, npm, or build step anywhere in this project. Everything — every public page, the booking API, and the admin panel — is one FastAPI app, run with `uvicorn app.main:app`.

One admin panel, served as a separate virtual host

Bookings and all content management (services, gallery, testimonials, FAQ, site settings) live behind a single login in `app/admin/` — a self-contained FastAPI app with its own top-level routes (`/`, `/login`, `/bookings`, `/services`, `/gallery`, `/testimonials`, `/faq`, `/settings`). It is mounted onto the main app via Starlette's `Host()` matcher (see `app/main.py`), keyed to `ADMIN_HOSTNAME` — `admin.localhost` locally (resolves to `127.0.0.1` in any modern browser, no `/etc/hosts` editing needed), a real subdomain (e.g. `admin.gpsusheringandevents.com`) in production. Because it's a distinct virtual host rather than a path prefix, every admin-side link and redirect is a plain top-level path that resolves correctly on whichever hostname it's reached through — no path-rewriting middleware needed.

There used to be a second, separate content-editing surface here (Decap CMS, a git-commit-based CMS at `/content-admin`, requiring a GitHub OAuth app). It was removed in favor of the single database-backed admin panel above — the business owner wanted full control without needing git/GitHub literacy, and one login covering everything is simpler to operate than two.

2. Project Structure

```
.
├── app/
│   ├── main.py          # Public FastAPI app: static mounts, routers, DB init, Host() mount for admin
│   ├── config.py        # pydantic-settings — reads env vars / .env
│   ├── database.py      # SQLAlchemy engine/session (sqlite:// or postgresql+psycopg://)
│   ├── models.py        # Booking, Service, Testimonial, GalleryItem, FAQItem, SiteSetting
│   ├── schemas.py       # Pydantic request/response schemas (bookings API)
│   ├── security.py      # Signed-cookie session helpers, shared by the whole admin panel
│   ├── content.py       # Reads the DB tables above -> plain dicts for the public templates
│   ├── seed.py          # One-time seed of the original starter content into an empty DB
│   └── routers/
│       ├── pages.py     # Every public page (/ , /about.html, ... /book-us.html)
│       └── bookings.py  # POST /api/bookings (public, honeypot-filtered)
│   └── admin/           # The whole admin panel, its own FastAPI app (see Section 1)
│       ├── __init__.py  # Builds admin_app, mounts its static/images, includes routers
│       └── routers/
│           ├── auth.py   # /login, /logout
│           ├── home.py   # / — dashboard landing with counts per section
│           ├── bookings.py # /bookings — list + status updates
│           ├── services.py # /services — full CRUD
│           ├── gallery.py # /gallery — full CRUD + image upload
│           ├── testimonials.py # /testimonials — full CRUD
│           ├── faq.py    # /faq — full CRUD
│           └── settings.py # /settings — single-row edit form
│   └── templates/
│       ├── base.html    # Public layout: head, topbar, navbar, footer, WhatsApp float
│       ├── partials/    # topbar.html, navbar.html, footer.html
│       ├── pages/       # index.html, about.html, services.html, ...
│       └── admin/       # login, home, bookings, and each content type's list/form templates
│   └── static/
│       ├── css/style.css # All public-site styling — CSS vars, light/dark theme tokens
│       ├── js/main.js    # All public-site client-side interactivity
│       ├── js/location-picker.js # Book Us page's Leaflet map picker
│       └── admin.css     # Admin-panel-only styling
├── images/uploads/      # Photos uploaded through the Gallery admin section land here
├── docs/                # Non-technical project docs (this file's siblings)
├── requirements.txt
├── Procfile             # `web: uvicorn app.main:app --host 0.0.0.0 --port $PORT`
└── .env.example
```

3. How a Page Request Actually Resolves

1. A route in `app/routers/pages.py` (e.g. `GET /services.html`) calls `content.get_services()`, which opens a short-lived DB session, queries the `services` table ordered by its `order` column, converts each row's `highlights` text blob into a list (split on newlines), and returns plain dicts.
2. The route builds a context dict via a shared `base_context(request, **extra)` helper (also in `pages.py`), which always injects `settings` (from the `site_settings` table) and `footer_services` (first 4 services, for the footer's link list).
3. `templates.TemplateResponse(request, "pages/services.html", context)` renders that template, which does `{% extends "base.html" %}` and fills in `{% block content %}`.
4. `base.html` pulls in the shared `partials/topbar.html`, `navbar.html`, `footer.html`, all reading from the same `settings` context variable.

Nothing is pre-built or cached — every request re-reads the database fresh. An admin panel save (add a testimonial, upload a gallery photo, edit a FAQ answer) is live on the very next public-page request, no rebuild step, no cache to bust. `app/seed.py` runs once at startup and only inserts the original starter content if a table is completely empty, so it never overwrites anything the business owner has since added or changed.

Admin panel auth

Single admin user, credentials from `ADMIN_USERNAME/ADMIN_PASSWORD` env vars, compared with `hmac.compare_digest`. On success, `app/security.py` issues an `itsdangerous`-signed cookie (no server-side session store — the cookie itself is the session, verified by signature + a 12-hour max age on every protected request). The same `require_admin` dependency guards every route under `app/admin/` — bookings and all content management alike. `COOKIE_SECURE` must be `false` for local `http://` testing (browsers silently drop a Secure cookie over plain HTTP) and should stay `true` in production.

The Book Us map picker

`app/static/js/location-picker.js` initializes a Leaflet map in `#location-picker` centered on Accra. Clicking the map or dragging its marker reverse-geocodes the picked coordinates via OpenStreetMap's free Nominatim API and writes the resulting address into the `#location` input — typing the address directly still works exactly as before, this is purely an assist. No API key, no billing account, and nothing server-side to configure.

Nominatim's usage policy asks for an identifying User-Agent or Referer for heavier automated use; browsers can't set a custom User-Agent from JS, but the page's Referer satisfies it for this kind of light, occasional client-side traffic. A site seeing heavy booking-form volume should move this behind a server-side proxy or a paid geocoding provider instead.

4. Data Model

TABLE	HOLDS
bookings	name, phone, email, event_type, event_date, guest_count, location, message, a status enum (new/contacted/confirmed/completed/cancelled), created_at
services	title, icon (Font Awesome class), order, home_description, description, highlights (newline-separated text)
testimonials	name, role, rating, order, quote
gallery_items	label, category, order, image (path under /images/uploads/, or empty)
faq_items	question, order, answer
site_settings	Single row (id=1) — site_name, site_url, tagline, phone_display, whatsapp_number, email, address, facebook/instagram/tiktok URLs

POST /api/bookings (app/routers/bookings.py) is public and unauthenticated by design — it's the public booking form's submit endpoint — but drops anything where the bot_field honeypot is filled in without storing or acknowledging it any differently to a bot. Every other table is only ever written to through the admin panel.

app/database.py rewrites a Railway-style postgres:// URL to postgresql+psycopg:// (psycopg 3) automatically; locally it defaults to sqlite:///./bookings.db with no setup required.

5. Running It Locally

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
ADMIN_USERNAME=admin ADMIN_PASSWORD=devpass SECRET_KEY=dev COOKIE_SECURE=false \
uvicorn app.main:app --reload --port 8000
```

Then `http://localhost:8000/` for the public site, and `http://admin.localhost:8000/` for the admin panel (login: admin/devpass) — bookings, services, gallery, testimonials, FAQ and settings all live there behind that one login.

6. Deployment Plan

Railway, one service, running the Procfile's `uvicorn app.main:app --host 0.0.0.0 --port $PORT`. Add a Postgres plugin (Railway injects `DATABASE_URL` automatically) — without it, bookings and any admin-panel edits live in a local SQLite file that won't survive a redeploy on Railway's ephemeral filesystem. Set the env vars from `.env.example`: `SECRET_KEY`, `ADMIN_USERNAME/ADMIN_PASSWORD`, `COOKIE_SECURE=true`, and `ADMIN_HOSTNAME` set to the real admin subdomain once its DNS is pointed at this service. **Cloudflare** is DNS-only in front of it (or skip it and point the domain straight at Railway).

Gallery photo uploads (`images/uploads/`) are also written to local disk. Like SQLite, these won't survive a redeploy on Railway's default ephemeral filesystem — attach a Railway volume mounted at `images/uploads/`, or move to object storage (e.g. Cloudflare R2), before real photos are uploaded in production.

7. Known Gaps / Worth Revisiting

- No automated tests yet — everything so far has been verified manually (curl + Playwright screenshots) rather than with a test suite.
- No pagination on the bookings list or any content list — fine at current volumes, will want it eventually.
- No rate limiting on POST /api/bookings beyond the honeypot field.

- Single hardcoded admin user — fine for one business owner, would need real user accounts to support more than one person.
- Gallery photo uploads and SQLite both need Railway's ephemeral-filesystem caveat addressed before going live (see the deployment callout above).
- `ADMIN_HOSTNAME` still defaults to `admin.localhost` — needs setting to the real subdomain (with matching DNS) before the admin panel is reachable in production.